Non-Parametric Regression and Trees

Econometría Aplicada y Ciencia de Datos - CIDE Fall 2026

Carlos Brito

Inspired by material from Francisco Cabrera:

`github.com/fcabrerahz/EconometricsDS`

Motivation

- Regression trees, or CART for Classification and Regression Trees.
  - They have reduced interpretability as **we do not observe a** β effect, but they are **useful to predict**.
  - Especially useful (for prediction) when there is a combination of continuous and discrete regressors.
- They are an **extension of econometric procedures**: e.g. a nonparametric regression using a large number of step functions. So we start with it.

Outline

- Non-parametric Regression
- Series and Spline Regression
- Regression Trees
- Bagging and Random Forest

Motivation: Why Go Beyond Linear Models?

So far we estimate $E[Y \mid X] = X'\beta$. But economic relationships are often nonlinear, and we rarely know their true shape.

Motivation: Why Go Beyond Linear Models?

So far we estimate $E[Y \mid X] = X'\beta$. But economic relationships are often nonlinear, and we rarely know their true shape.

Examples:

- **Wages and experience (Mincer):** earnings rise fast early in the career, then flatten. Is a quadratic in experience really the right shape?

Motivation: Why Go Beyond Linear Models?

So far we estimate $E[Y \mid X] = X'\beta$. But economic relationships are often nonlinear, and we rarely know their true shape.

Examples:

- **Wages and experience (Mincer):** earnings rise fast early in the career, then flatten. Is a quadratic in experience really the right shape?
- **Engel curves:** the share of spending on food falls with income, but not necessarily linearly

Motivation: Why Go Beyond Linear Models?

So far we estimate $E[Y \mid X] = X'\beta$. But economic relationships are often nonlinear, and we rarely know their true shape.

Examples:

- **Wages and experience (Mincer):** earnings rise fast early in the career, then flatten. Is a quadratic in experience really the right shape?
- **Engel curves:** the share of spending on food falls with income, but not necessarily linearly
- **Regression discontinuity:** to estimate the effect of a program at an eligibility cutoff, we fit flexible local regressions on each side

Motivation: Why Go Beyond Linear Models?

So far we estimate $E[Y \mid X] = X'\beta$. But economic relationships are often nonlinear, and we rarely know their true shape.

Examples:

- **Wages and experience (Mincer):** earnings rise fast early in the career, then flatten. Is a quadratic in experience really the right shape?
- **Engel curves:** the share of spending on food falls with income, but not necessarily linearly
- **Regression discontinuity:** to estimate the effect of a program at an eligibility cutoff, we fit flexible local regressions on each side

$\Rightarrow$ **Nonparametric regression** lets the data choose the shape of $m(x) = E[Y \mid X = x]$.

Non-Parametric Regression

- Nonparametric estimation of the conditional expectation function:

$$\mathbb{E}[Y \mid X = x] = m(x)$$

- An economic model restricts the form of $m(x)$ to a parametric function. But if $m(x)$ can take any nonlinear shape it is therefore **nonparametric**.

- **We will discuss nonparametric kernel smoothing estimators of $m(x)$.**

Non-Parametric Regression

Consider a single real-valued regressor X.

- The nonparametric regression model is:

$$Y = m(X) + e, \qquad \mathbb{E}[e \mid X] = 0, \qquad \mathbb{E}[e^2 \mid X] = \sigma^2(X).$$

Non-Parametric Regression

Consider a single real-valued regressor X.

- The nonparametric regression model is:

$$Y = m(X) + e, \qquad \mathbb{E}[e \mid X] = 0, \qquad \mathbb{E}[e^2 \mid X] = \sigma^2(X).$$

- We have n observations for the pair (Y, X).
- The goal is to estimate $m(x)$ either at a single point x or at a set of points.
  - For most of the theory, we focus on estimation at a single point x.

Non-Parametric Regression

Consider a single real-valued regressor X.

- The nonparametric regression model is:

$$Y = m(X) + e, \qquad \mathbb{E}[e \mid X] = 0, \qquad \mathbb{E}[e^2 \mid X] = \sigma^2(X).$$

- We have n observations for the pair (Y, X).
- The goal is to estimate $m(x)$ either at a single point x or at a set of points.
  - For most of the theory, we focus on estimation at a single point x.
- In addition to the Gauss–Markov assumptions, we assume that both $m(x)$ and $f(x)$ (the marginal density of X) are continuous in x.

Binned Means Estimator

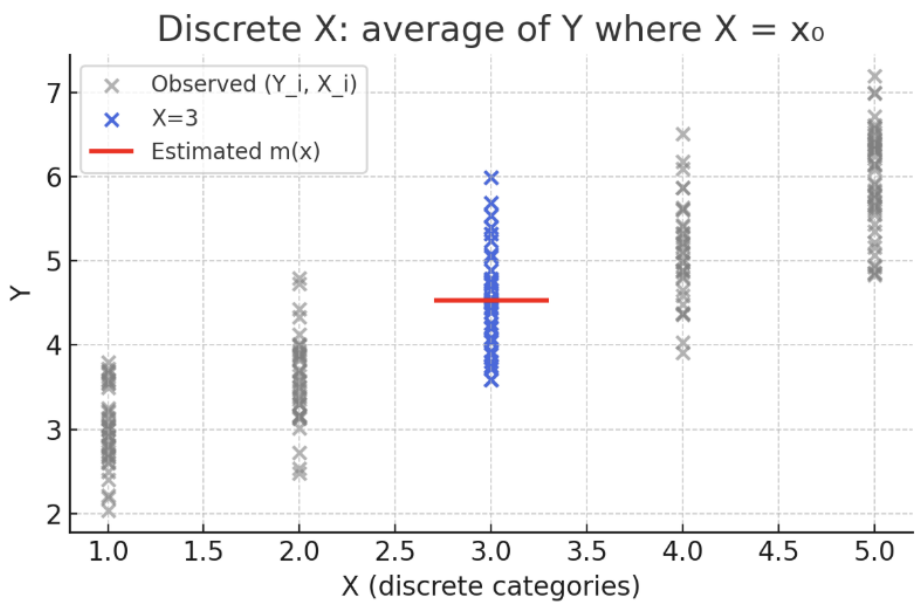
Fix the point x and consider estimation of $m(x)$:

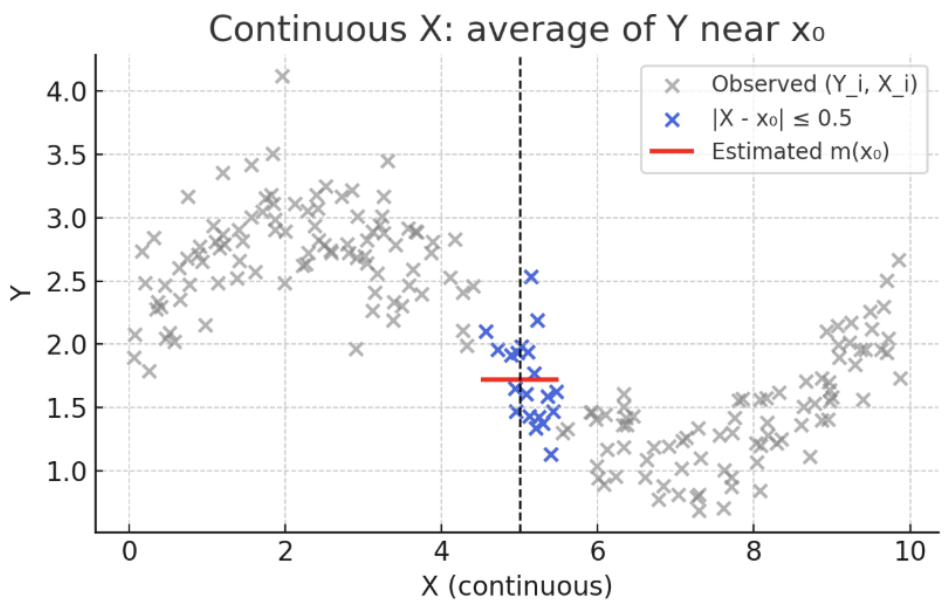
- We get the mean of Y for random pairs (Y, X) such that $X = x$.
  - X discrete: we take the average of the sub-sample of observations Y_i with $X_i = x$.
  - X continuous: we take the average of the observations for which X_i is close to x. Say: $|X_i - x| \leq h$ for some small $h \geq 0$. h is the bandwidth.

Binned Means Estimator

Fix the point x and consider estimation of $m(x)$:

- We get the mean of Y for random pairs (Y, X) such that $X = x$.
  - X discrete: we take the average of the sub-sample of observations Y_i with $X_i = x$.
  - X continuous: we take the average of the observations for which X_i is close to x. Say: $|X_i - x| \leq h$ for some small $h \geq 0$. h is the bandwidth.





Notes: Discrete X: we compute $m(x) = \mathbb{E}[Y \mid X = x]$ by taking the mean of Y for all observations where $X_i = x$. For continuous X we estimate $m(x)$ using the average of Y values for observations where X_i is close to x (within a chosen bandwidth).

Binned Means Estimator

The **nonparametric estimator** of $m(x)$ using a local (uniform) window of half-width h is

$$\hat{m}(x) = \frac{\sum_{i=1}^{n} 1\{|X_i - x| \leq h\} Y_i}{\sum_{i=1}^{n} 1\{|X_i - x| \leq h\}}$$

- **Numerator:** sum of Y_i for observations inside the window $[x - h, x + h]$.
- **Denominator:** number of observations inside the window.
- So $\hat{m}(x)$ is simply the **average of Y among observations close to x**.

Binned Means Estimator

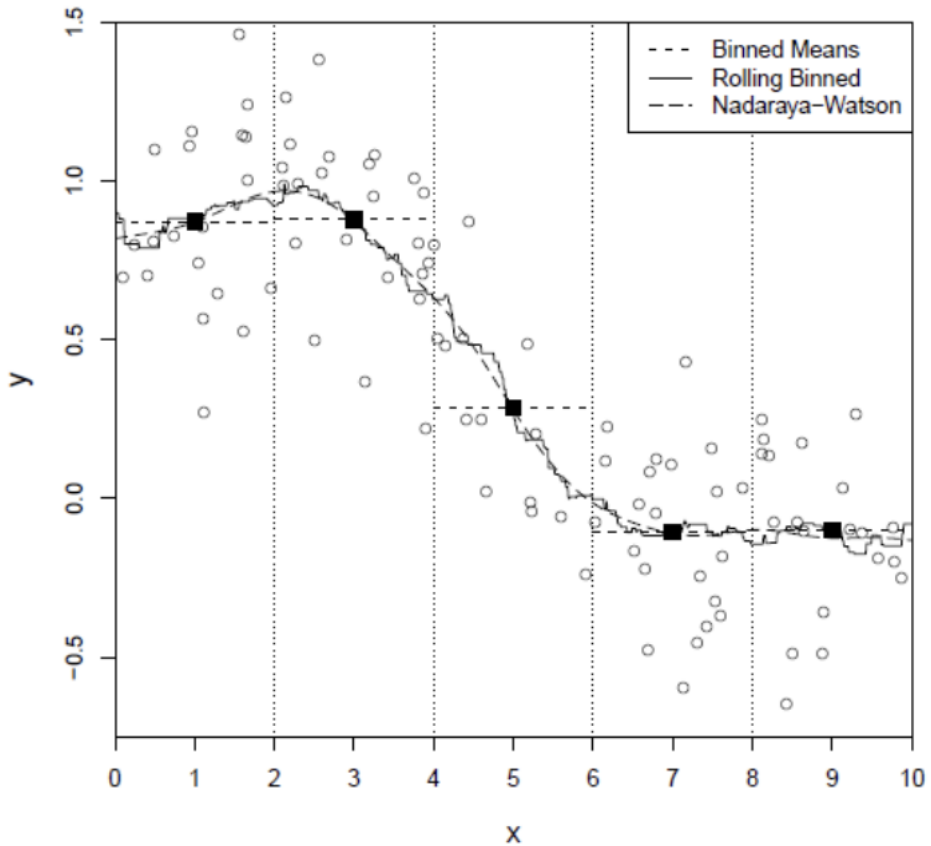
The **nonparametric estimator** of $m(x)$ using a local (uniform) window of half-width h is

$$\hat{m}(x) = \frac{\sum_{i=1}^{n} 1\{|X_i - x| \leq h\} Y_i}{\sum_{i=1}^{n} 1\{|X_i - x| \leq h\}}$$

- **Numerator:** sum of Y_i for observations inside the window $[x - h, x + h]$.
- **Denominator:** number of observations inside the window.
- So $\hat{m}(x)$ is simply the **average of Y among observations close to x**.

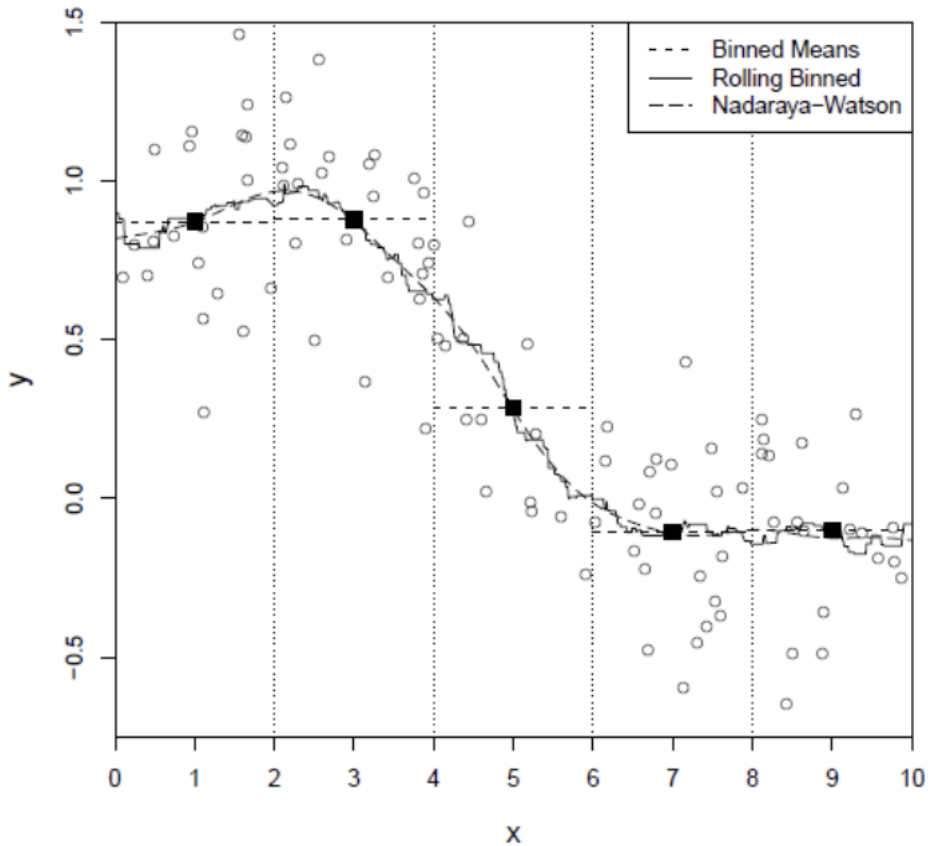
This is a **step function** estimator of $m(x)$: as x moves, the estimate jumps whenever an observation enters or leaves the window.

Binned Means Estimator



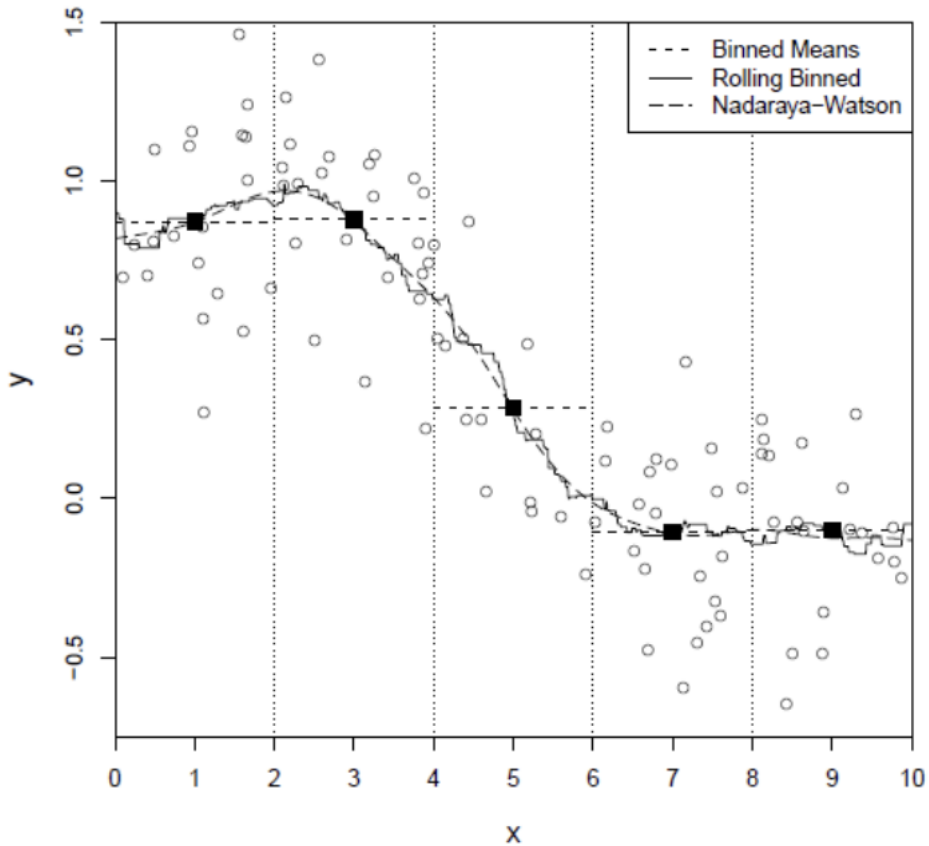
- In panel (a): one flat average per bin.
- The estimate jumps at the edges of each bin.

Binned Means Estimator



- In panel (a): one flat average per bin.
- The estimate jumps at the edges of each bin.

Binned Means Estimator



- In panel (a): one flat average per bin.
- The estimate jumps at the edges of each bin.
- **Smooth weights:** give observations closer to x more weight, and let the weight fade gradually (kernel regression, local linear regression). This produces a smooth $\hat{m}(x)$. $\Rightarrow$ Next slides.

Kernel Regression

- The source of the discontinuity in the step function is that the weights are discontinuous indicator functions:
  - i.e. the function $1\{|X_i - x| \leq h\}$ assigns equal weight $= 1$ to all points in the window $[x - h, x + h]$ and 0 to all others.

Kernel Regression

- The source of the discontinuity in the step function is that the weights are discontinuous indicator functions:
  - i.e. the function $1\{|X_i - x| \leq h\}$ assigns equal weight $= 1$ to all points in the window $[x - h, x + h]$ and 0 to all others.
- If instead the weights are continuous functions: $\hat{m}(x)$ will also be continuous in x.

Kernel Regression

- The source of the discontinuity in the step function is that the weights are discontinuous indicator functions:
  - i.e. the function $1\{|X_i - x| \leq h\}$ assigns equal weight $= 1$ to all points in the window $[x - h, x + h]$ and 0 to all others.
- If instead the weights are continuous functions: $\hat{m}(x)$ will also be continuous in x.
- e.g. the **Nadaraya–Watson** estimator uses a smooth kernel function $K(\cdot)$, which gives more weight to points closer to x and vice versa.
  - A kernel function is a bounded probability density function which is symmetric about zero satisfying:

$$\int_{-\infty}^{\infty} u^2 K(u)\, du = 1.$$

Nadaraya–Watson Estimator

The Nadaraya–Watson method estimates $m(x) = \mathbb{E}[Y|X = x]$ as a **weighted local average** of nearby Y_i values:

$$\hat{m}(x) = \frac{\sum_{i=1}^{n} K\left(\frac{X_i - x}{h}\right) Y_i}{\sum_{i=1}^{n} K\left(\frac{X_i - x}{h}\right)}$$

- $X_i - x$ measures how close observation i is to the point of interest.
- $K(\cdot)$: kernel function (smooth, symmetric, positive) ensures scale invariance.
- h: bandwidth (controls smoothness). The larger, the smoother.
- **Kernels**: Rectangular, **Gaussian**, Epanechnikov, Triangular.
- Generalizes the discrete-window estimator by assigning **smooth weights** that decline with distance from x.

Local Linear Regression (LLR)

- The Nadaraya–Watson estimator estimates a local constant that works well if $m(x)$ is roughly constant within the bandwidth h. **The local slope is ignored!**

- The local linear estimator fits a local line at each point x:

$$\min_{a,b} \sum_{i=1}^{n} \left(Y_i - a - b(X_i - x)\right)^2 K\left(\frac{X_i - x}{h}\right).$$

- If $b = 0$:

Local Linear Regression (LLR)

- The Nadaraya–Watson estimator estimates a local constant that works well if $m(x)$ is roughly constant within the bandwidth h. **The local slope is ignored!**

- The local linear estimator fits a local line at each point x:

$$\min_{a,b} \sum_{i=1}^{n} \left(Y_i - a - b(X_i - x)\right)^2 K\left(\frac{X_i - x}{h}\right).$$

- If $b = 0$: we get the minimization problem of Nadaraya–Watson.

Local Linear Regression (LLR)

- The Nadaraya–Watson estimator estimates a local constant that works well if $m(x)$ is roughly constant within the bandwidth h. **The local slope is ignored!**

- The local linear estimator fits a local line at each point x:

$$\min_{a,b} \sum_{i=1}^{n} \left(Y_i - a - b(X_i - x)\right)^2 K\left(\frac{X_i - x}{h}\right).$$

- If $b = 0$: we get the minimization problem of Nadaraya–Watson.
- if $h \to \infty$:

Local Linear Regression (LLR)

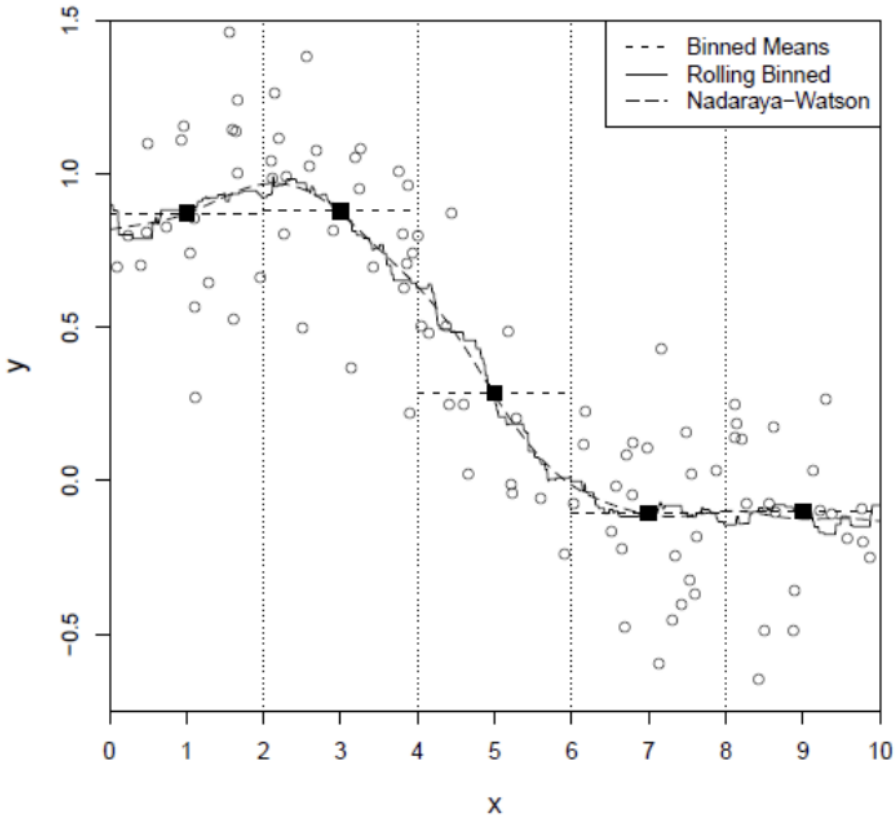
- The Nadaraya–Watson estimator estimates a local constant that works well if $m(x)$ is roughly constant within the bandwidth h. **The local slope is ignored!**

- The local linear estimator fits a local line at each point x:

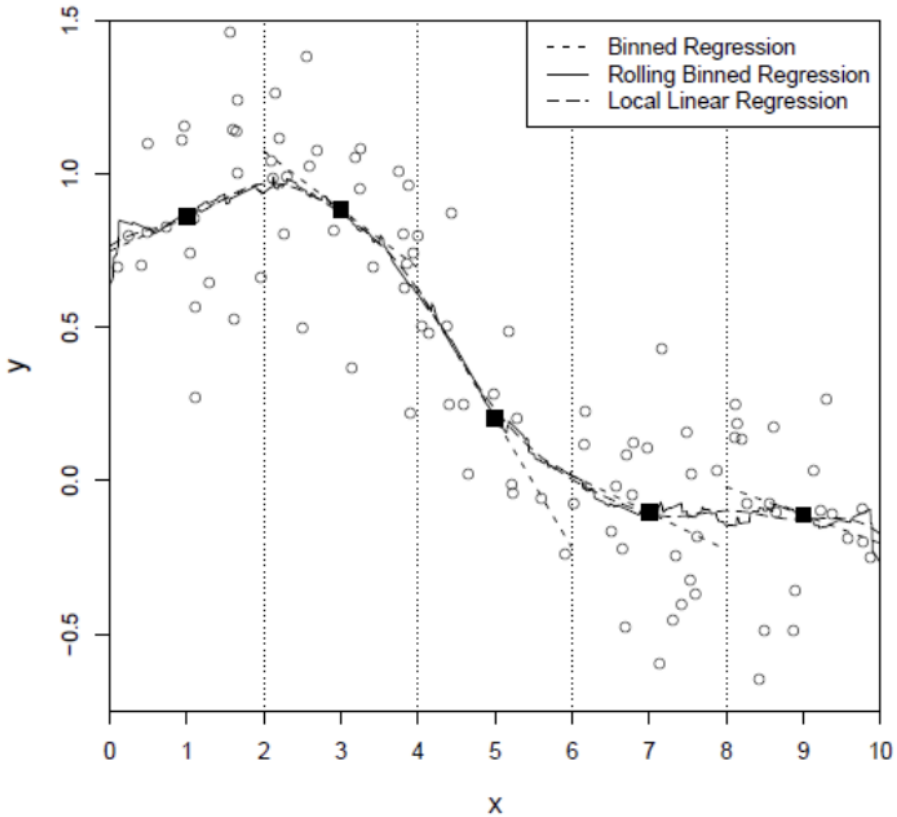
$$\min_{a,b} \sum_{i=1}^{n} \left(Y_i - a - b(X_i - x)\right)^2 K\left(\frac{X_i - x}{h}\right).$$

- If $b = 0$: we get the minimization problem of Nadaraya–Watson.
- if $h \to \infty$: we simply get the OLS estimator including all observations with equal weight.
- Hence it provides estimators of the slope at x.

Nadaraya–Watson Estimator and Local Linear Regression



(a) Nadaraya-Watson



(b) Local Linear

Local Polynomial Estimator

We approximate $m(X_i)$ near x with a polynomial of degree p:

$$m(X_i) \approx a_0 + a_1(X_i - x) + a_2(X_i - x)^2 + \cdots + a_p(X_i - x)^p.$$

Estimate the coefficients $(a_0, \ldots, a_p)$ by:

$$\min_{a_0, \ldots, a_p} \sum_{i=1}^{n} \left[Y_i - \sum_{j=0}^{p} a_j (X_i - x)^j \right]^2 K\left(\frac{X_i - x}{h} \right).$$

The fitted value at x is $\hat{m}_{LP}(x) = \hat{a}_0$.

- Higher $p \rightarrow$ lower bias, higher variance due to more parameters.
- Usually $p = 1$ (local linear) or $p = 2$ (local quadratic) in practice.

The Bandwidth

The bandwidth h determines how *local* the nonparametric regression is.

Bandwidth	Effect	Bias	Variance
Small h	Very local fit; follows noise	Low bias	High variance
Large h	Very smooth fit; averages widely	High bias	Low variance

The bandwidth controls the **bias–variance tradeoff**.

Rule of Thumb (ROT) for NW

A **rule-of-thumb bandwidth** gives a practical starting point based on theoretical approximations.

For a Gaussian kernel and roughly normal X:

$$h_{\text{rot}} = 1.06\, n^{-1/5}\, \hat{\sigma}_X$$

- Larger samples: smaller h (more local detail).
- More dispersed X: larger h (wider smoothing window).
- It provides a reasonable default but does not adapt to the actual data pattern.
- LLR computation or ROT explained in Hansen (see p. 676).

Cross-Validation (Data-Driven)

Cross-validation (CV) chooses h by minimizing out-of-sample prediction error.

Leave-one-out CV criterion:

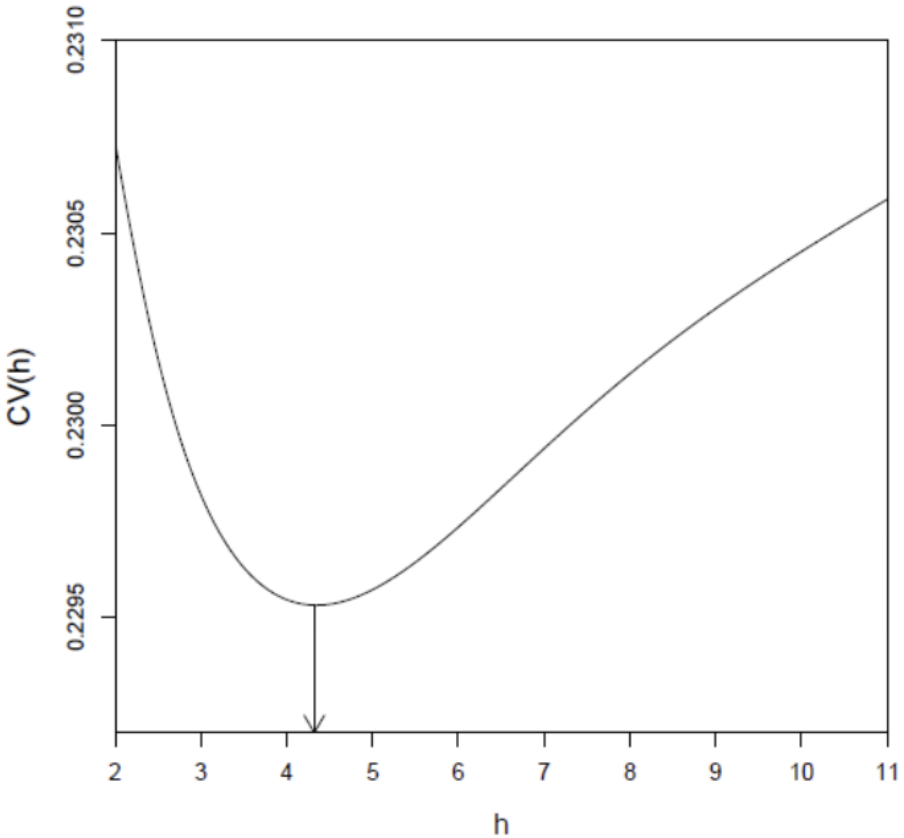
$$CV(h) = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{m}_{-i}(X_i; h))^2$$

Where $\hat{m}_{-i}(X_i; h)$ is estimated without observation i.

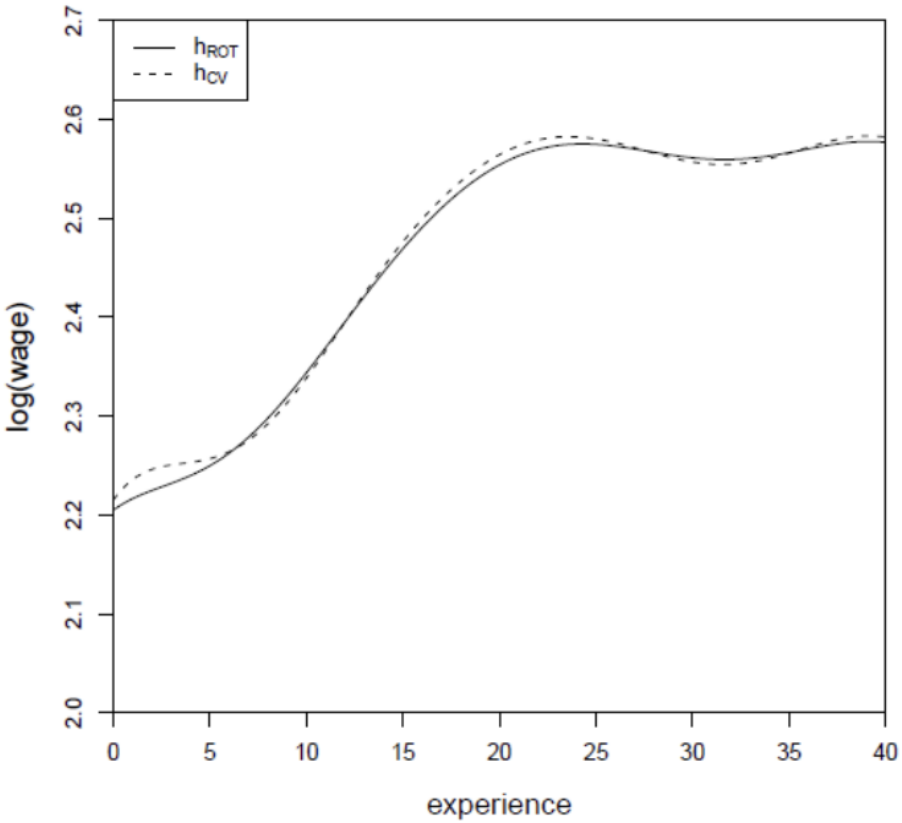
As described before:

- Pick a grid of h values.
- Compute $CV(h)$ for each (leaving one out — fold or observation).
- Choose $\hat{h}$ that minimizes $CV(h)$.

Mincer Non-Parametric Regression



(a) Cross-Validation Criterion



(b) Local Linear Regression

Figure 19.4: log(*wage*) regression on *experience*

Conditional Variance

Note that LLR can be expressed with regressor vector $Z_i(x) : Z = [X_i - x]$ and coefficient vector $\beta(x)$.

- Applying weighted least squares with the kernel weights we obtain the LL estimator:

$$\widehat{\beta}_{LL}(x) = \left(\sum_{i=1}^{n} K\left(\frac{X_i - x}{h} \right) Z_i(x) Z_i(x)' \right)^{-1} \sum_{i=1}^{n} K\left(\frac{X_i - x}{h} \right) Z_i(x) Y_i$$

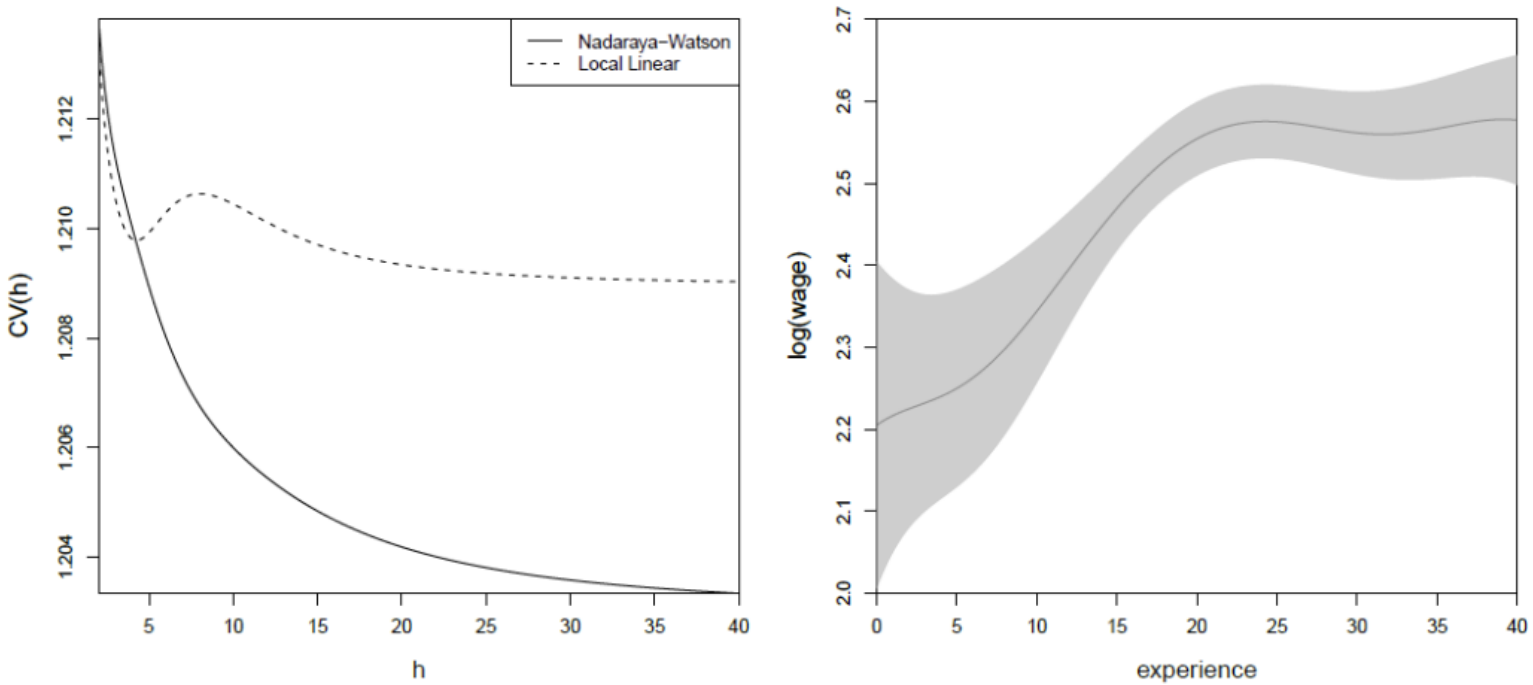
$$= (Z' K Z)^{-1} Z' K Y$$

- where: $\mathrm{K} = \mathrm{diag}\{K((X_1 - x)/h), \ldots, K((X_n - x)/h)\}$
- Notice that the matrices Z and K depend on x and h.

Conditional Variance

In this course we do not study in detail conditional nor asymptotic variance, but note that it is also a weighted "sandwich":

$$\widehat{V}_{\hat{\beta}}(x) = (Z'KZ)^{-1}\left(\sum_{i=1}^{n} K\left(\frac{X_i - x}{h}\right)^2 Z_i(x)Z_i(x)'\hat{e}_i^2\right)(Z'KZ)^{-1}.$$



(a) Cross-Validation for Conditional Variance (b) Regression with Confidence Bands

Figure 19.5: Confidence Band Construction

R code!

Outline

- Non-parametric Regression
- Series and Spline Regression
- Regression Trees
- Bagging and Random Forest

An alternative class of nonparametric regression.

- A series regression model is a sequence $K = 1, 2, \ldots$ of approximating models $m_K(x)$ with K parameters.
- Approximating models could be polynomial or *splines*.
- Linear series regression models take the form

$$Y = X_K' \beta_K + e_K$$

- Where $X_K = X_K(X)$ is a vector of regressors obtained by making transformations of X, and β_K is a coefficient vector.

Series Regression

Instead of assuming a simple linear model:

$$m(x) = \beta_0 + \beta_1 X$$

we represent $m(x)$ as a **series expansion** that allows nonlinear shapes:

$$m_K(x) = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_p X^p.$$

Here:

- $K = p + 1$ is the number of basis functions (constant + p polynomial terms).
- Increasing K makes the model more flexible.
- The series regression approximates $m(x)$ by fitting a weighted (by β_p) combination of these basis functions.

We build a **design matrix** X_K that contains transformations of X:

$$X_K = \begin{bmatrix} 1 & X_1 & X_1^2 & \dots & X_1^p \\ 1 & X_2 & X_2^2 & \dots & X_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & X_n & X_n^2 & \dots & X_n^p \end{bmatrix}.$$

Then we estimate the coefficients by OLS:

$$\hat{\beta}_K = (X_K' X_K)^{-1} X_K' Y.$$

- That is, we run a single regression of Y on all polynomial terms of X.
- Each column in X_K represents one basis function (e.g., 1, X, X^2).
- The estimator of $m(x)$ is $\hat{m}_k(x) = X_K(x)' \hat{\beta}_k$.

It is called the series regression because it is obtained by adding the series of variable X^p.

Spline Regression

A **spline** is a piecewise polynomial.

- The flexibility of the model is determined by the number of polynomial segments.
- The join points between segments are called **knots**.
- To impose smoothness and parsimony we constrain the spline function to have continuous derivatives up to the order of the spline.
- Typically the order of the polynomial is pre-selected to be linear, quadratic, or cubic.

Spline Regression

A **linear spline** with one knot τ is

$$m_K(x) = \beta_0 + \beta_1 x + \beta_2 (x - \tau)\, 1\{x \geq \tau\}.$$

- For $x \leq \tau$ the function $m_K(x) = \beta_0 + \beta_1 x$ is linear with slope β_1;
- For $x \geq \tau$ the function $m_K(x)$ is linear with slope $\beta_1 + \beta_2$;
- Note that β_2 is the change in the slope at τ.

A **linear spline** with two knots $\tau_1 < \tau_2$ is

$$m_K(x) = \beta_0 + \beta_1 x + \beta_2 (x - \tau_1)\, 1\{x \geq \tau_1\} + \beta_3 (x - \tau_2)\, 1\{x \geq \tau_2\}.$$

A **quadratic spline** with one knot is

$$m_K(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 (x - \tau)^2 \, 1\{x \geq \tau\}.$$

- Observe that for $x \leq \tau$ the function is the quadratic $\beta_0 + \beta_1 x + \beta_2 x^2$ with second derivative $m''_K(\tau) = 2\beta_2$;
- For $x \geq \tau$ the second derivative is $m''_K(\tau) = 2(\beta_2 + \beta_3)$; so $2\beta_3$ is the change in the second derivative at τ.
- The first derivative at $x = \tau$ is the continuous function $m'_K(\tau) = \beta_1 + 2\beta_2 \tau$.

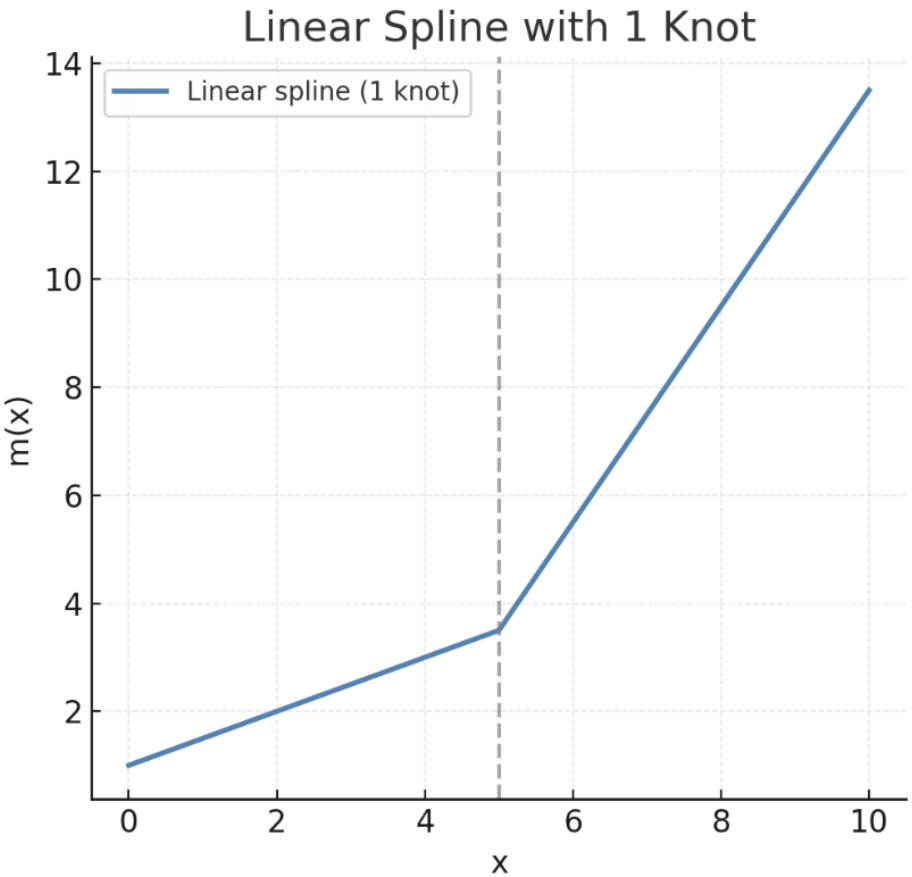
Ensuring continuity at the knot.

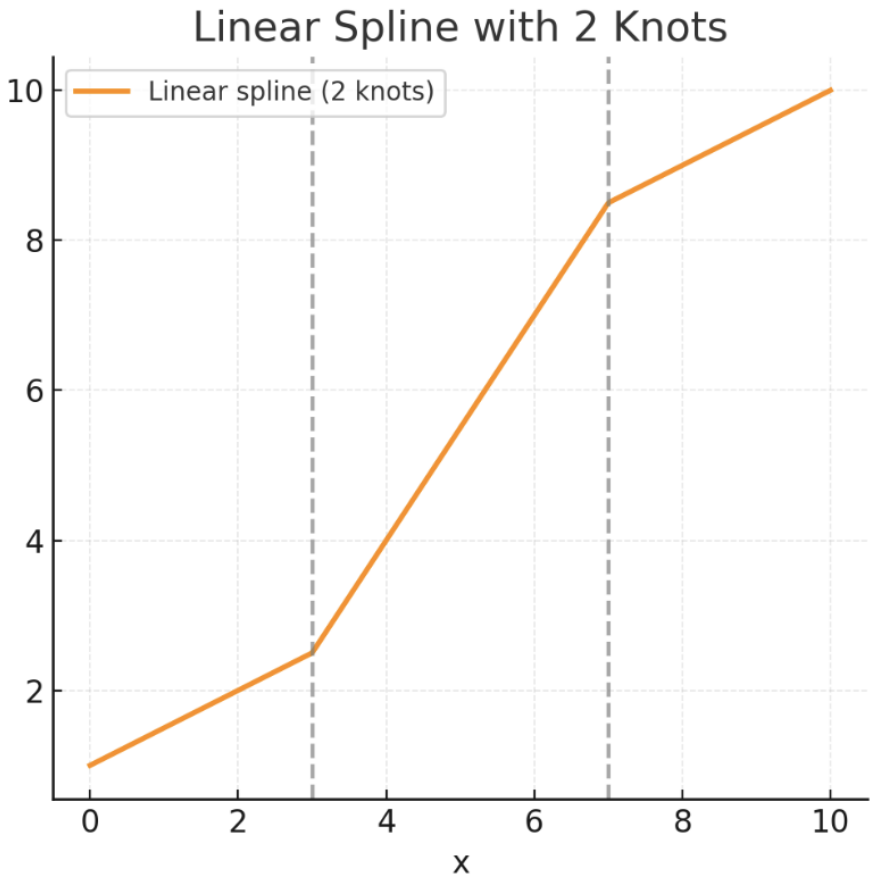
Spline Regression

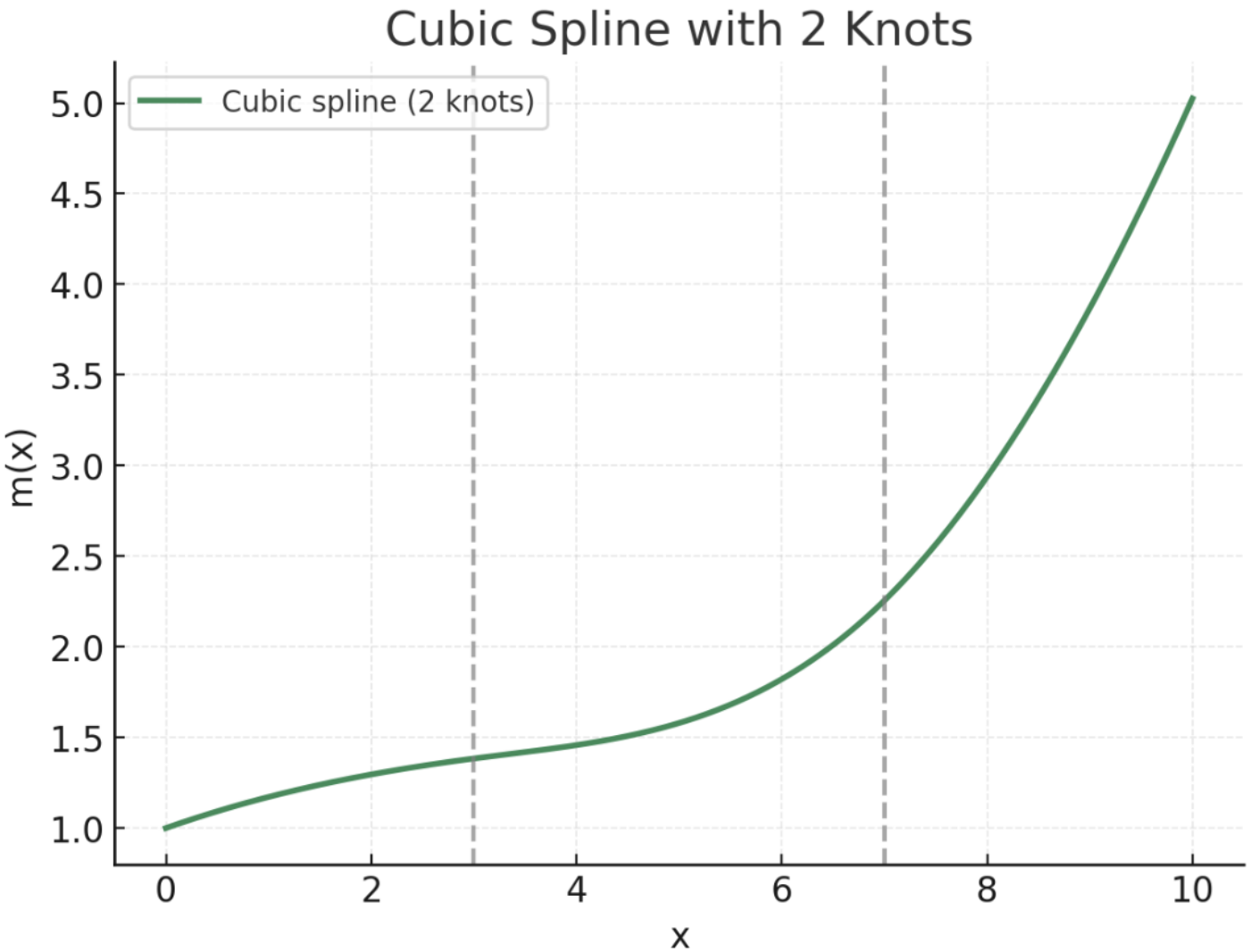
In general, a pth-order spline with N knots $\tau_1 < \tau_2 < \cdots < \tau_N$ is

$$m_K(x) = \sum_{j=0}^{p} \beta_j x^j + \sum_{k=1}^{N} \beta_{p+k} (x - \tau_k)^p \, 1\{x \geq \tau_k\}$$

Splines


Linear Spline with 1 Knot
Linear spline (1 knot)
m(x)
x
0
2
4
6
8
10
12
14


Linear Spline with 2 Knots
Linear spline (2 knots)
x
0
2
4
6
8
10

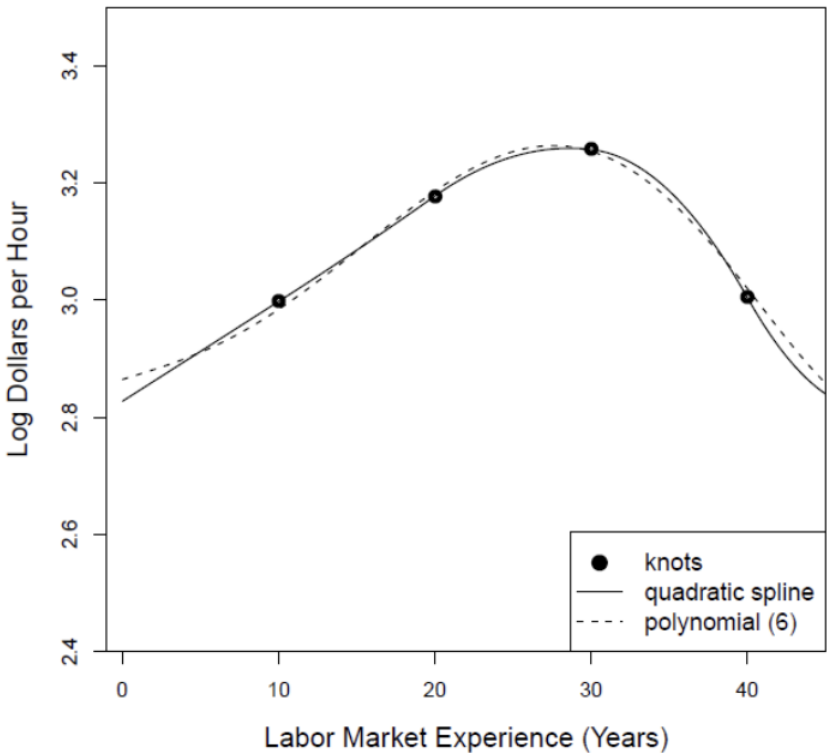


Cubic spline with two knots — smoother and continuously differentiable across segments.

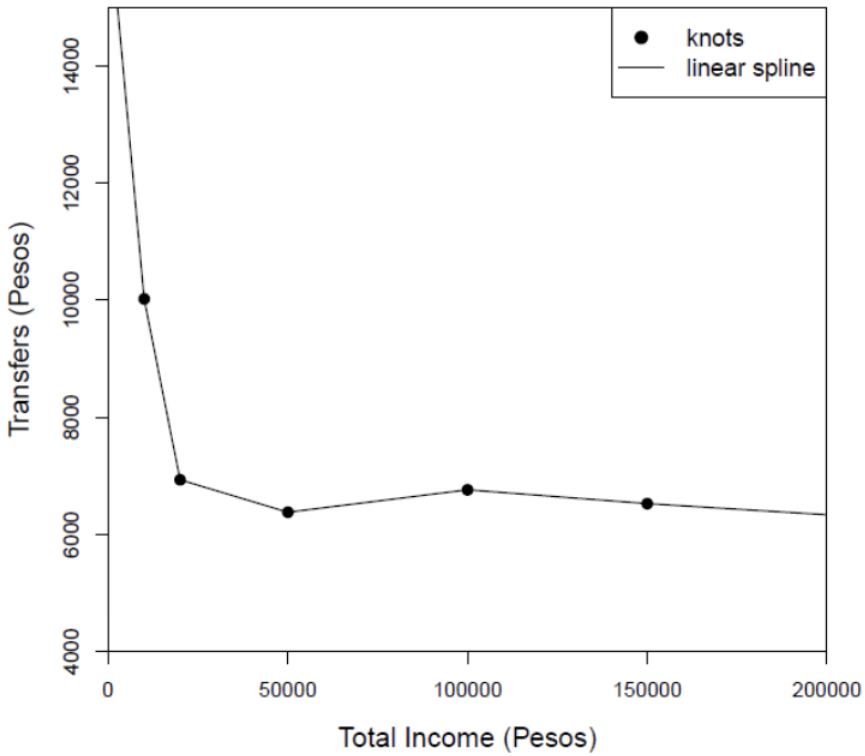
Splines

- In practice a spline will depend critically on the choice of the knots τ_k.
- When X has an approximately **uniform distribution** it is common to space the knots **evenly** so all segments have the same length.
- When the distribution of X is not uniform, an alternative is to set the knots at the quantiles $j/(N+1)$.
- A third alternative is to set the knots at the points where $m(x)$ has the greatest change in curvature.

Regression Spline Example



(a) Experience Profile



(b) Effect of Income on Transfers

Figure 20.2: Spline Regression Estimates

Outline

- Non-parametric Regression
- Series and Spline Regression
- Regression Trees
- Bagging and Random Forest

Regression Trees

CARTs (Classification and Regression Trees):

- A regression tree is a nonparametric regression using a **large number of step functions**.
- With a large number of split points a step function can approximate any function.

When there is a large combination of continuous and discrete regressors:

- Kernel and series methods are challenging to implement.
- Regression trees can be thought of as a 0th-order spline with **free knots**.

Regression Trees

Estimate $m(x) = E[Y \mid X = x]$ for scalar Y and vector X. The elements of X can be continuous, binary, or ordinal.

- If a regressor is categorical it should first be transformed to a set of binary variables.

Language based on the metaphor of a living tree:

- A subsample is a **branch**.
- Terminal branches are **nodes** or **leaves**.
- Increasing the number of branches is **growing** a tree.
- Decreasing the number of branches is **pruning** a tree.

Regression Trees

- The basic algorithm starts with a single branch.
- Grow a large tree by sequentially splitting the branches. Then **prune back** using an information criterion.
- The goal of the growth stage is to develop a rich, data-determined tree with small estimation bias.
- Pruning back has the goal of reducing **over-parameterization** and estimation variance (backward stepwise regression).

Regression Trees

At the beginning of the regression tree algorithm:

- The entire dataset is treated as one branch (no splits).
- The predicted value is the **overall mean**:

$$\hat{m}(x) = \mu$$

Then the regression tree algorithm makes use of a **regression sample split algorithm**.

- The method uses non-linear least squares (NLLS) to estimate the model:

$$Y = \mu_1 1\{X_d \leq \gamma\} + \mu_2 1\{X_d > \gamma\} + e$$

- Linear in μ_1, μ_2, but **nonlinear in the cutoff** γ.

Regression Trees

- The "Machine" chooses the cut (γ) that makes the two groups differ the most while reducing the overall SSR.
- We estimate μ_j and the split γ to **minimize** $SSR(\gamma)$.

$$SSR(\gamma) = \sum_{i:X_{di} \leq \gamma} (Y_i - \bar{Y}_L)^2 + \sum_{i:X_{di} > \gamma} (Y_i - \bar{Y}_R)^2$$

Grow a Tree

For a fixed size of the leaf (final node: N_{min}):

(a) Apply the regression sample split algorithm to split each branch into two sub-branches, with size at least N_{min}.

(b) On each sub-branch, choose the branch that minimizes:

$$\text{SSR}(\gamma) = \sum (Y_i - \hat{m}(X_d; \gamma))^2$$

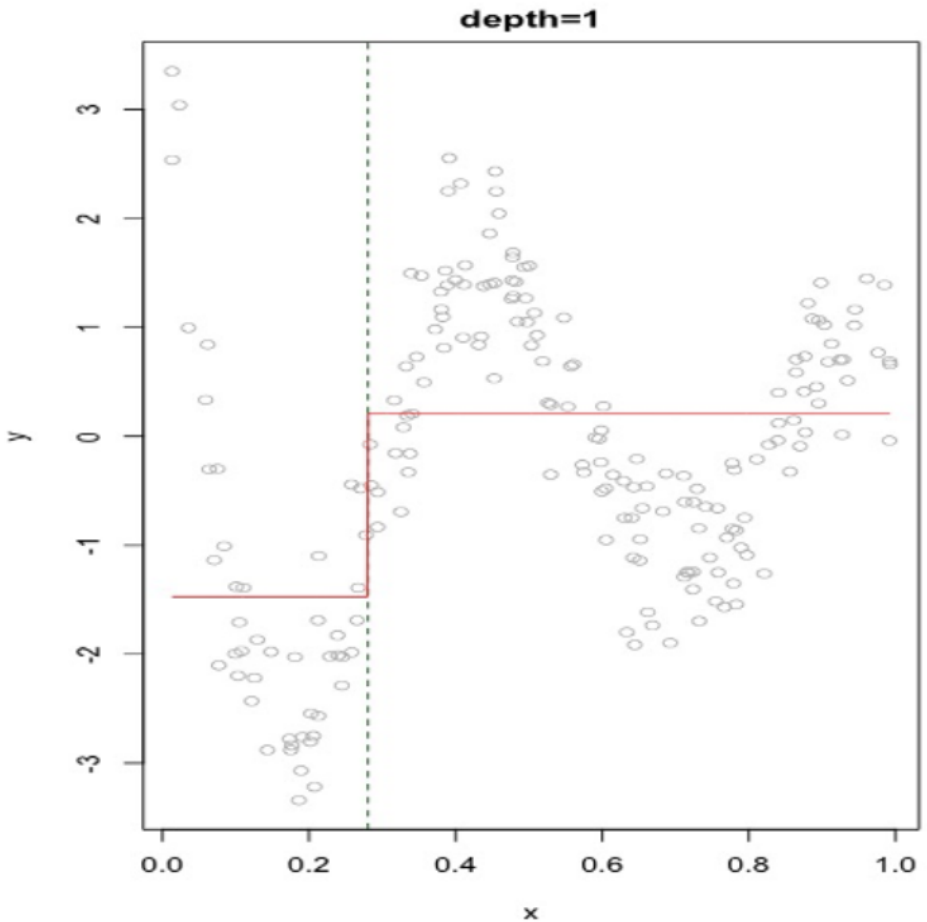
(c) Split again such branch into two branches.

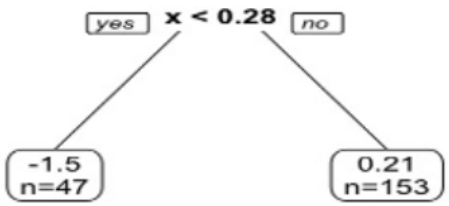
Repeat **(a)–(b)** until each branch cannot be further split (due to N_{min}).

Regression Trees

ML finds the best split which minimizes the deviations to the mean in each leaf.

- This is a tree of depth 1:





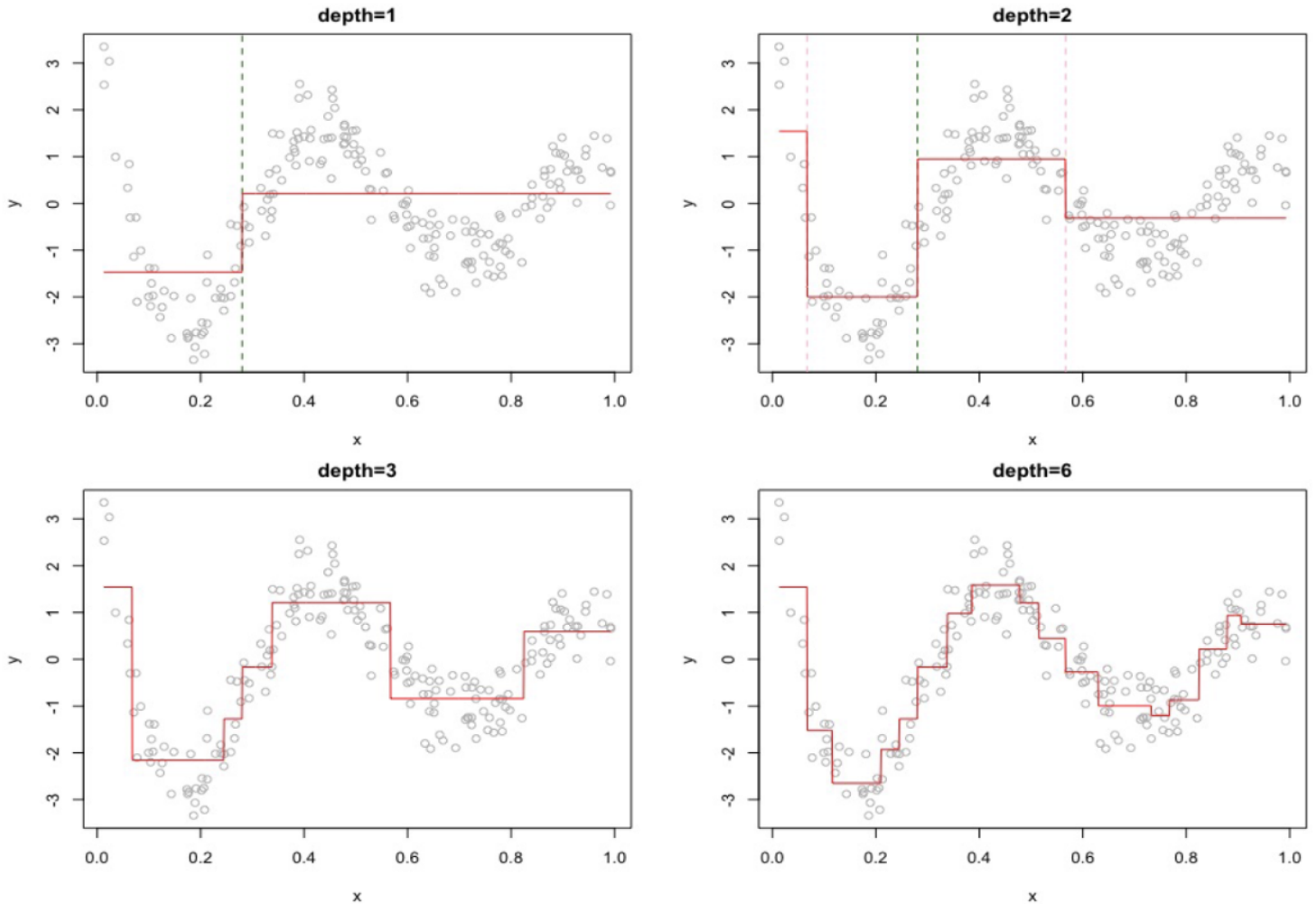
Grow a Tree

- The terminal (unsplit) branches are the **leaves**.
- Before that: adding subsequent branches improves prediction capacity.
- e.g. Two branches: step function with one cutoff. Many branches: piecewise-constant approximation (a "grown" tree).
- Each split **reduces prediction error** and increases flexibility.

Regression Trees

ML finds the best split which minimizes the deviations to the mean in each leaf.

- These are trees with higher depths:



Prune a Tree

Define the *Mallows-type* information criterion (that penalizes model complexity or N leaves):

$$C = \sum_{i=1}^{n} \hat{e}_i^2 + \alpha N$$

- where N is the number of leaves and α is a penalty parameter (similar to a λ in non-parametric regression).

Prune a Tree

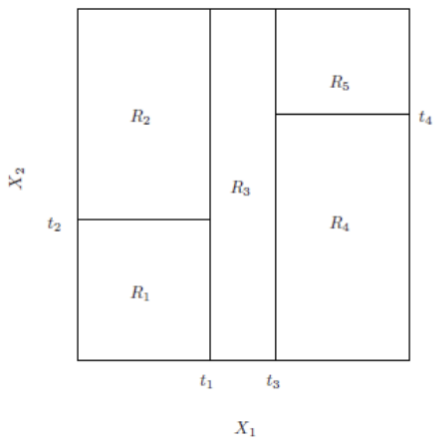
- Compute the criterion C for the current tree.
- **Use backward stepwise regression to reduce the number of leaves:**

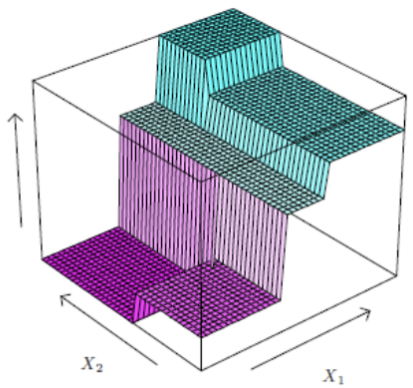
(a) Identify the leaf whose removal most decreases C.
(b) Prune (remove) this leaf.
(c) If there is no leaf whose removal decreases C, then stop pruning.
(d) Otherwise, repeat **(a)–(c)**.

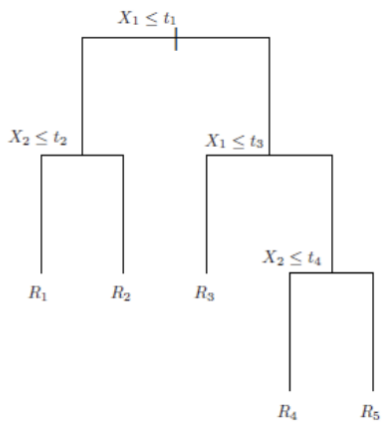
The penalty parameter α is selected by K-fold cross-validation. A smaller tree with fewer splits might lead to lower variance and better interpretation at the cost of a little bias.

Regression Trees

This is a tree with three covariates:







Disadvantages:

- The results are difficult to interpret as there are no regression coefficients.
- Fitted regression $\hat{m}(x)$ is a discrete step function, a crude approximation when $m(x)$ is continuous and smooth.
- A regression tree may require a high number of leaves, which can result in a non-parsimonious model with high estimation variance.

Regression Tree Code

Tree vs. Linear Model

Depending on the problem at hand:

- **Linear regression:**

$$m(X) = \beta_0 + \sum_{j=1}^{K} X_j \beta_j$$

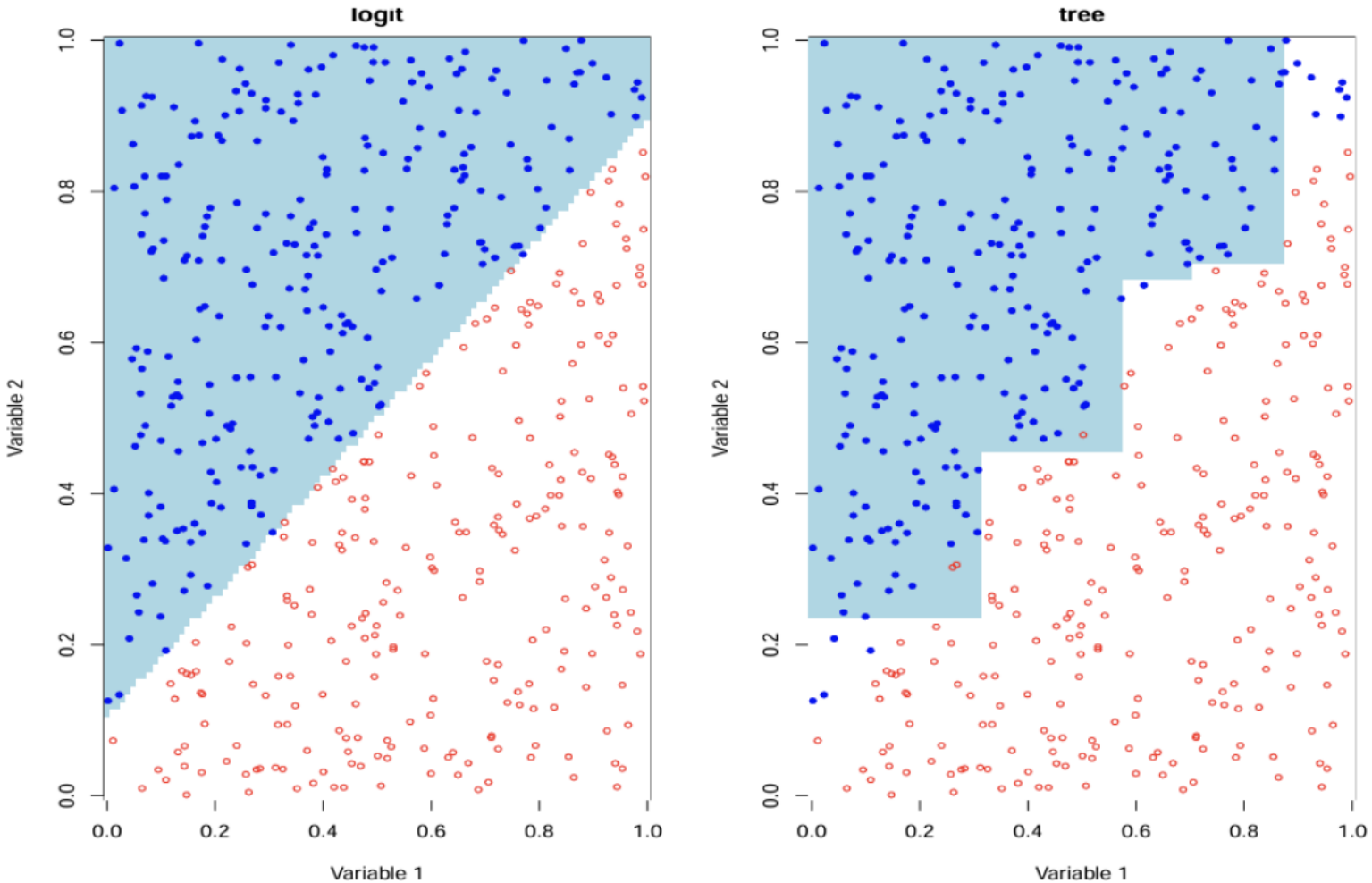
- **Regression tree:**

$$m(X) = \sum_{j=1}^{J} c_j 1(x \in R_j)$$

- If the relationship between y and $(x_1, \ldots, x_K)$ is **linear**, a linear model should perform better.
- If the relationship between y and $(x_1, \ldots, x_K)$ is **highly nonlinear and complex**, a tree model should perform better.

A Classification Tree

This is a tree with three covariates:



Outline

- Non-parametric Regression
- Series and Spline Regression
- Regression Trees
- Bagging and Random Forest

Bagging

Bagging (bootstrap aggregating) is a method to reduce the variance of a predictor.

(1) Generate a large number B of bootstrap samples (with replacement).
(2) Estimate your regression model on each bootstrap sample, and take the average of the estimates.

The mean of the bootstrap estimates is the **bagging estimator** of the conditional mean.

Bagging

Let $m(x) = \mathbb{E}[Y \mid X = x]$ be the conditional mean and $\hat{m}(x)$ an estimator such as a regression tree. Let $\hat{m}_b^*(x)$ be the same estimator constructed on a bootstrap sample.

- The **bagging** estimator of $m(x)$ is:

$$\hat{m}_{\mathbf{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{m}_b^*(x).$$

- As B increases this converges in bootstrap probability to the ideal bagging estimator $\mathbb{E}^*[\hat{m}^*(x)]$.

Bagging

- Useful when the conditional mean estimator has low bias but high variance.
- Useful for hard threshold estimators such as regression trees.
- Each tree uses around 2/3 of the observations.
- Due to replacement, some observations will appear more than once and some will never appear.
- The remaining 1/3 observations are referred to as the **out-of-bag (OOB)** observations, used for prediction and mean squared error (MSE).

Bagging

- Each regression tree might split the data differently depending on which points get resampled.
- Some trees will split earlier, others will be smoother.
- When you average them, the random idiosyncrasies cancel out, leaving a smoother and more stable prediction function.

Bagging may exaggerate bias:

- Bootstrap smooths extremes — what if these are important predictors?
- Averaging can pull predictions toward the center, which leads to *more* bias.

Gimme the code!

Random Forests

- Bootstrap samples are similar to one another, hence the estimated bootstrap regression trees will also be similar.
- e.g. Bagging will create many trees where the first splits will be given by the main predictor (education, in the code example).
- This causes collinearity across trees.
- Variance of the bootstrap average remains high even when the number of bootstrap replications B is large.
- Random forests decorrelate the bootstrap regression trees by adding randomness, reducing MSE.
- Random forests have effectively displaced simple regression trees.

Random Forests

The basic algorithm follows:

(1) Leaf size N_{min} (default $= 5$), split fraction $\alpha \in [0, 1)$, and sampling number $m < p$ (default $= p/3$).

(2) For $b = 1, \ldots, B$: grow a regression tree on a bootstrap sample using:

- m variables at random from the p regressors.
- Among m, pick one producing the best regression split with N_{min} and at least α of n in the branch.
- Split the bootstrap sample accordingly.
- Stop when each leaf has between N_{min} and $2N_{\text{min}} - 1$ observations.

(3) Set $\hat{m}_b(x)$ as the sample mean of Y on each leaf: $\hat{m}_{\text{rf}}(x) = B^{-1} \sum_{b=1}^{B} \hat{m}_b(x)$.

Random Forests

Using randomization to reduce the number of variables from p to m at each step alters the tree structure.

- Reduces the correlation between the bootstrapped regression trees.
- While random forests are popular in applications, **a distributional theory has been slow to develop**.

*Note: **Honest trees**, developed by Wager and Athey (2019) — beyond the scope of this course — suggest viable properties of asymptotic normality.*

Random Forest Code

"Real life" application